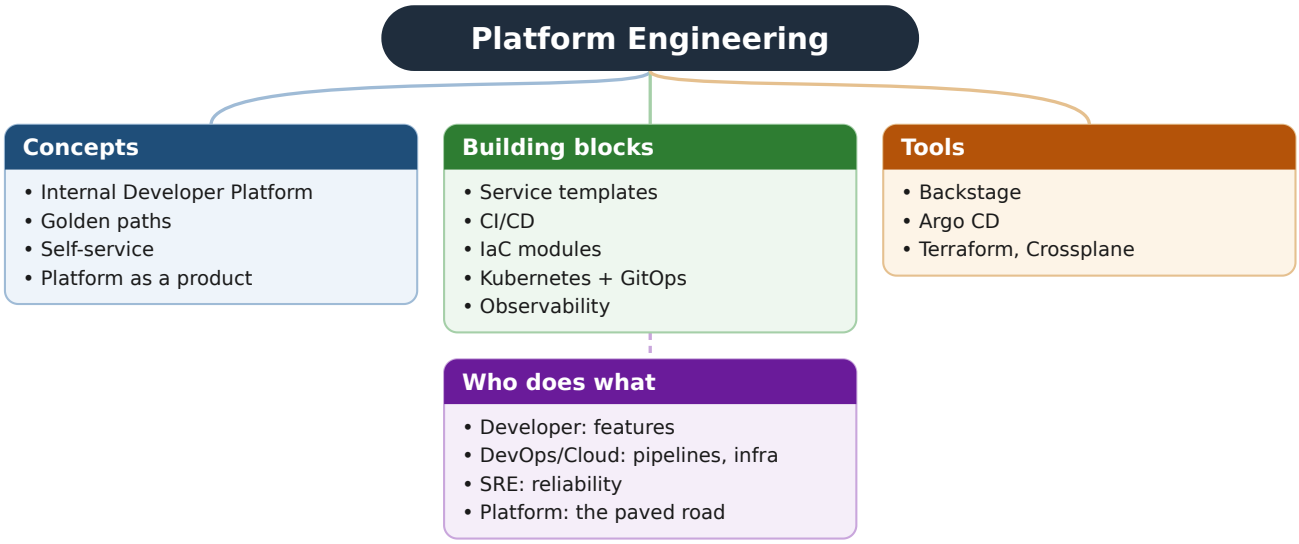
16. Platform Engineering

DevOps, Cloud & Platform Engineering · Complete Notes · Topic 16 of 18 · Thirupathi Keloth · v2.0

Contents

16.0 Why platform engineering
16.1 Core concepts
16.2 Platform capabilities
16.3 Golden paths
16.4 Backstage: the developer portal
16.5 Self-service infrastructure
16.6 Environments and multi-tenancy
16.7 Designing platform abstractions
16.8 Governance, security and cost by default
16.9 Measuring the platform
16.10 Reference architecture: an IDP on AWS
16.11 Building a platform: approach and anti-patterns
16.12 Hands-on labs
16.13 Interview questions
16.14 Self-check



16.0 Why platform engineering

"You build it, you run it" gave teams ownership, but also handed every developer Kubernetes, Terraform, IAM, CI pipelines, observability, security scanners and cost management. The result in many companies: **cognitive overload**, every team reinventing the same pipelines slightly differently, inconsistent security, and senior engineers spending their time on plumbing.

Platform engineering builds an **Internal Developer Platform (IDP)**: a curated, self-service layer of tools, workflows and infrastructure that lets application teams ship and run software **without becoming infrastructure experts**, while the organization gets standardization, security and cost control by default. It is the natural career growth path for DevOps engineers.

16.1 Core concepts

Term	Meaning
Internal Developer Platform (IDP)	The whole set of self-service capabilities (infrastructure, CI/CD, environments, observability, security) glued together for developers

Term	Meaning
Internal developer portal	The front door / UI of the platform (e.g. Backstage): catalog, templates, docs, links. A portal is **not** the platform; it's one interface to it
Platform as a product	The platform team treats developers as customers: user research, roadmap, documentation, support, adoption metrics. Not a ticket queue
Golden path / paved road	The opinionated, supported, well-documented way to do a common task (create a service, add a database), fast and compliant by default. Optional, but so convenient most teams choose it
Self-service	Developers get what they need through a portal, CLI, API or Git PR, in minutes, without filing tickets
Cognitive load	The mental effort a team spends; the platform absorbs the *extraneous* load (infra plumbing) so teams focus on their product
Thinnest viable platform	Start with the smallest platform that removes real pain (sometimes just docs + templates + one pipeline); grow from demand
Escape hatch	A supported way to go off the golden path when there's a real need, with clear responsibility

Team Topologies (the organizational model behind platforms)

Team type	Purpose
Stream-aligned	Delivers business value end-to-end (checkout team, search team)
Platform	Provides internal services that reduce stream-aligned teams' cognitive load
Enabling	Experts who coach teams to adopt new skills (e.g. SRE practices, security), then move on
Complicated-subsystem	Owns a part needing deep specialist knowledge (video codec, ML model serving)

Interaction modes: **X-as-a-Service** (the goal for a mature platform: consume without talking to the platform team), **Collaboration** (working closely for a period, e.g. building a new capability with a pilot team), **Facilitating** (enabling teams teaching).

DevOps vs SRE vs Platform Engineering

	DevOps	SRE	Platform Engineering
What	Culture and practices: collaboration, automation, ownership	Reliability engineering with SLOs and error budgets	Building internal products for developer self-service
Main output	Automated delivery, shared responsibility	Reliable production, incident practice	The IDP, golden paths
Relationship	The philosophy	One implementation of operations	One implementation of DevOps at scale

They coexist: a platform team provides the paved road (often including SRE tooling), stream-aligned teams own their services on it.

16.2 Platform capabilities

Based on the CNCF Platforms white paper, a complete platform provides these capabilities through consistent interfaces:

Capability	Examples of what developers get
Infrastructure provisioning	A database, queue, bucket or cache via template/PR/API (Terraform modules, Crossplane, ACK)
Compute and runtime	Kubernetes namespaces with quotas, or serverless/containers (EKS, ECS, Lambda)
Application environments	dev/staging/prod plus ephemeral preview environments per PR
Build, test, delivery	Reusable CI workflows, GitOps deployment, progressive delivery
Observability	Dashboards, logs, traces and SLO alerts automatically for every new service
Security	Scanning in pipelines, signed images, policies, identity (Pod Identity/IRSA), secret store integration
Secrets and configuration	ExternalSecrets wired to Secrets Manager; config per environment
Messaging and data	Approved Kafka/SQS/RDS patterns with backup and encryption
Identity and access	SSO, team-based RBAC, least-privilege roles created automatically
Developer portal and docs	Service catalog, ownership, API docs, TechDocs, scorecards
Cost visibility	Cost per service/team (tags, Kubecost/OpenCost), budgets and alerts

Interfaces: portal (UI forms), CLI, API, and **Git** (a PR to a config repo is often the best self-service API: reviewed, audited, GitOps-applied).

16.3 Golden paths

A golden path for "create a new backend service" produces, in minutes:

```
developer fills a template form (name, team, language, needs a database? public API?)
   │
   ▼ scaffolder
1. Git repository from a template: app skeleton, Dockerfile, tests, README, CODEOWNERS,
   catalog-info.yaml, reusable CI workflow reference, Dependabot config
2. Config repo PR: Kustomize/Helm values for dev/staging/prod, Argo CD Application
3. Infrastructure claim: e.g. PostgreSQL (Crossplane claim / Terraform module PR)
4. Secrets wiring: ExternalSecret pointing at the generated database credentials
5. Observability: ServiceMonitor, default dashboard, default SLO + burn-rate alerts
6. Security: image scanning, signing and policy checks inherited from the shared CI workflow
7. Catalog registration: owner, system, links (repo, CI, Argo CD, dashboards, runbooks)
   │
   ▼
first deployment to dev runs automatically: "time to first deploy" measured in minutes
```

Properties of a good golden path: optional but attractive; opinionated defaults (security, observability) built in; documented end to end; maintained (templates updated centrally, existing services upgraded through reusable workflows/modules rather than copy-paste); measured (adoption, time saved).

Reusable building blocks carry updates to all services: reusable CI workflows (`uses: acme/platform/.github/workflows/build.yml@v3`), Helm library charts, Terraform modules with versions, Kyverno policies, base images.

16.4 Backstage: the developer portal

Backstage (created by Spotify, CNCF incubating) is the most widely adopted open-source portal framework. Core features:

Feature	What it does
Software Catalog	Inventory of all software: services, websites, libraries, APIs, resources, with owners and relationships, read from `catalog-info.yaml` files in repos
Software Templates (Scaffolder)	Forms that create repos, PRs, and call APIs: the golden path engine
TechDocs	Docs-as-code (Markdown in the repo with MkDocs) rendered in the portal
Search	Across catalog, docs, and plugins
Plugins	Kubernetes, Argo CD, GitHub Actions, PagerDuty, Grafana, SonarQube, cost, security… shown on each entity's page
Scorecards (via plugins)	Production readiness checks per service (has owner, runbook, SLO, no critical CVEs)

Catalog entities

```
# catalog-info.yaml in the shop-api repository
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: shop-api
  description: Orders and checkout API
  annotations:
    github.com/project-slug: acme/shop-api
    backstage.io/techdocs-ref: dir:.
    argocd/app-name: shop-api-prod
    backstage.io/kubernetes-id: shop-api
    pagerduty.com/service-id: P1234AB
  tags: [python, fastapi]
  links:
    - url: https://grafana.example.com/d/shop-api
      title: Dashboard
    - url: https://wiki.example.com/runbooks/shop-api
      title: Runbook
spec:
  type: service
  lifecycle: production
  owner: group:shop-team
  system: shop
  providesApis: [shop-orders-api]
  dependsOn:
    - resource:shop-orders-db
---
apiVersion: backstage.io/v1alpha1
kind: API
metadata:
  name: shop-orders-api
spec:
  type: openapi
  lifecycle: production
  owner: group:shop-team
  system: shop
  definition:
    $text: ./openapi.yaml
---
apiVersion: backstage.io/v1alpha1
kind: Resource
metadata:
  name: shop-orders-db
spec:
  type: database
  owner: group:shop-team
  system: shop
```

Entity kinds: `Component` (service, website, library), `API`, `Resource` (database, bucket), `System` (group of components), `Domain` (business area), `Group` and `User` (usually synced from the identity provider), `Template`, `Location`.

Software template (scaffolder)

```
apiVersion: scaffolder.backstage.io/v1beta3
kind: Template
metadata:
  name: python-service
  title: Python service (golden path)
  description: FastAPI service with CI, GitOps deployment, observability and security defaults
spec:
  owner: group:platform-team
  type: service
  parameters:
    - title: Service details
      required: [name, owner]
      properties:
        name:
          type: string
          pattern: "^[a-z][a-z0-9-]{2,30}$"
          description: Lowercase name, e.g. orders-api
        owner:
          type: string
          ui:field: OwnerPicker
          ui:options:
            catalogFilter:
              kind: Group
        database:
          type: boolean
          default: false
          title: Needs a PostgreSQL database
  steps:
    - id: fetch
      name: Generate the repository content
      action: fetch:template
      input:
        url: ./skeleton
        values:
          name: ${{ parameters.name }}
          owner: ${{ parameters.owner }}
          database: ${{ parameters.database }}
    - id: publish
      name: Create the GitHub repository
      action: publish:github
      input:
        repoUrl: github.com?owner=acme&repo=${{ parameters.name }}
        defaultBranch: main
        protectDefaultBranch: true
    - id: register
      name: Register in the catalog
      action: catalog:register
      input:
        repoContentsUrl: ${{ steps.publish.output.repoContentsUrl }}
        catalogInfoPath: /catalog-info.yaml
  output:
    links:
      - title: Repository
        url: ${{ steps.publish.output.remoteUrl }}
      - title: Open in catalog
        icon: catalog
        entityRef: ${{ steps.register.output.entityRef }}
```

Further steps typically open a PR in the GitOps repo (`publish:github:pull-request`) adding the service's Argo CD Application and, if `database` is true, a database claim.

Running Backstage is real engineering: it's a TypeScript/React app you build and deploy yourself (plugins are code). Alternatives: **Roadie** (hosted Backstage), **Port**, **Cortex**, **OpsLevel**, **Atlassian**

Compass: less flexible, much less maintenance.

16.5 Self-service infrastructure

Approach	How developers request	Notes
Terraform modules + PR	Add a module block (from the internal registry) in an infra repo; CI plans, reviewers approve, Atlantis/HCP Terraform/Spacelift applies	Most common; familiar; review-based
Crossplane	Apply a simple Kubernetes object (a claim/composite resource) in their namespace; Crossplane creates cloud resources and keeps reconciling them	Kubernetes-native; GitOps-friendly; drift corrected continuously
AWS Controllers for Kubernetes (ACK)	Kubernetes CRDs mapping 1:1 to AWS resources	Low abstraction; often used inside Crossplane-like compositions or by platform teams
Portal action calling an API	Template step triggers a pipeline	Good UX; the pipeline still uses one of the above

Crossplane in brief

```
Platform team defines:  CompositeResourceDefinition (XRD) = the API
                          ("PostgreSQLInstance" with size, version)
                        Composition = how to build it on AWS (RDS instance, subnet group,
                          security group, secret) using providers (provider-aws) and functions
Developer applies:      a small resource of that kind in their namespace ⟶ Crossplane creates
                        everything and writes connection details into a Secret for the app
```

```
# The platform API: what developers may set (size and version only)
apiVersion: apiextensions.crossplane.io/v1
kind: CompositeResourceDefinition
metadata:
  name: xpostgresqlinstances.platform.acme.io
spec:
  group: platform.acme.io
  names:
    kind: XPostgreSQLInstance
    plural: xpostgresqlinstances
  claimNames:
    kind: PostgreSQLInstance
    plural: postgresqlinstances
  versions:
    - name: v1alpha1
      served: true
      referenceable: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                size:
                  type: string
                  enum: [small, medium, large]   # mapped to instance classes by the platform
                version:
                  type: string
                  enum: ["16", "17"]
              required: [size]
```

```
# What a developer writes (in the GitOps repo, applied by Argo CD)
apiVersion: platform.acme.io/v1alpha1
kind: PostgreSQLInstance
metadata:
  name: orders-db
  namespace: shop
spec:
  size: small
  version: "16"
  writeConnectionSecretToRef:
    name: orders-db-conn                    # host, port, username, password for the app
```

The Composition (owned by the platform team) maps `small` to `db.t4g.medium`, forces encryption, private subnets, backups, deletion protection in prod and tags for cost: developers can't get it wrong. Crossplane **v2** (2025) makes composite resources namespaced directly (claims become optional/legacy) and builds compositions from **functions** pipelines; the concepts stay the same.

16.6 Environments and multi-tenancy

Model	Isolation	Cost / overhead	Use
Namespace per team/app (with RBAC, quotas, NetworkPolicies, Pod Security)	Soft	Lowest	Most internal workloads
Virtual clusters (**vcluster**)	Each team gets its own API server inside a namespace	Low–medium	Teams needing CRDs/cluster-admin-like freedom, CI test clusters

Model	Isolation	Cost / overhead	Use
Cluster per environment	Strong between dev/staging/prod	Medium	Standard baseline
Cluster per team / per tenant	Strongest	Highest	Regulated or untrusted tenants
Account per environment/workload (AWS)	Hard boundary for IAM, billing, quotas	Managed by landing zone	AWS best practice

Ephemeral preview environments: each PR gets a namespace (or vcluster) with the app deployed (ApplicationSet pull-request generator), a URL posted on the PR, and automatic deletion on merge/close. Seed data and stubbed dependencies keep them cheap; TTL controllers prevent leaks.

16.7 Designing platform abstractions

The art is choosing **what to hide and what to expose**:

Too low (leaky)	Right level	Too high (rigid)
Developers write 400 lines of Kubernetes YAML and IAM JSON	Developers declare intent: image, port, CPU/memory size, public or internal, needs Postgres; platform generates the rest with secure defaults	A single "deploy" button that can't express health checks, scaling or a second container

Patterns:

- **App specification** owned by developers, rendered by the platform: a Helm umbrella chart with a small values file, a Kustomize component library, a CUE/KCL schema, or **Score** (a workload spec translated to Compose/Kubernetes/Helm).
- **Sensible defaults + documented overrides** (escape hatches) for the 10% of cases that need them.
- **Version your platform APIs** (`v1alpha1` → `v1`) and give migration paths; never break hundreds of services silently.

```
# Example developer-facing app spec (rendered by the platform's chart/templates)
service: orders-api
team: shop
image: 123456789012.dkr.ecr.ap-south-1.amazonaws.com/shop/orders-api
port: 8080
size: medium                  # platform maps to requests/limits and HPA bounds
exposure: public              # public -> Ingress + WAF + TLS; internal -> ClusterIP only
healthPath: /healthz
slo:
  availability: 99.9
dependencies:
  postgres: orders-db
```

16.8 Governance, security and cost by default

Concern	Platform mechanism
Security	Shared pipelines with scanning/signing, Kyverno/OPA policies, Pod Security `restricted` namespaces, workload identity provisioned automatically
Compliance	Policies as code, audit trails from Git and CI, scorecards for production readiness

Concern	Platform mechanism
Reliability	Default probes, PDBs, spread constraints, SLOs and alerts in templates
Cost (FinOps)	Mandatory tags/labels (`team` , `service` , `env`) enforced by policy; showback dashboards (Kubecost/OpenCost, AWS Cost Explorer by tag); budgets and anomaly alerts; right-sizing recommendations (VPA, Compute Optimizer); auto-shutdown of non-prod at night; spot/Graviton by default where safe
Lifecycle	Deprecation of old runtimes and versions tracked in the catalog; automated upgrade PRs

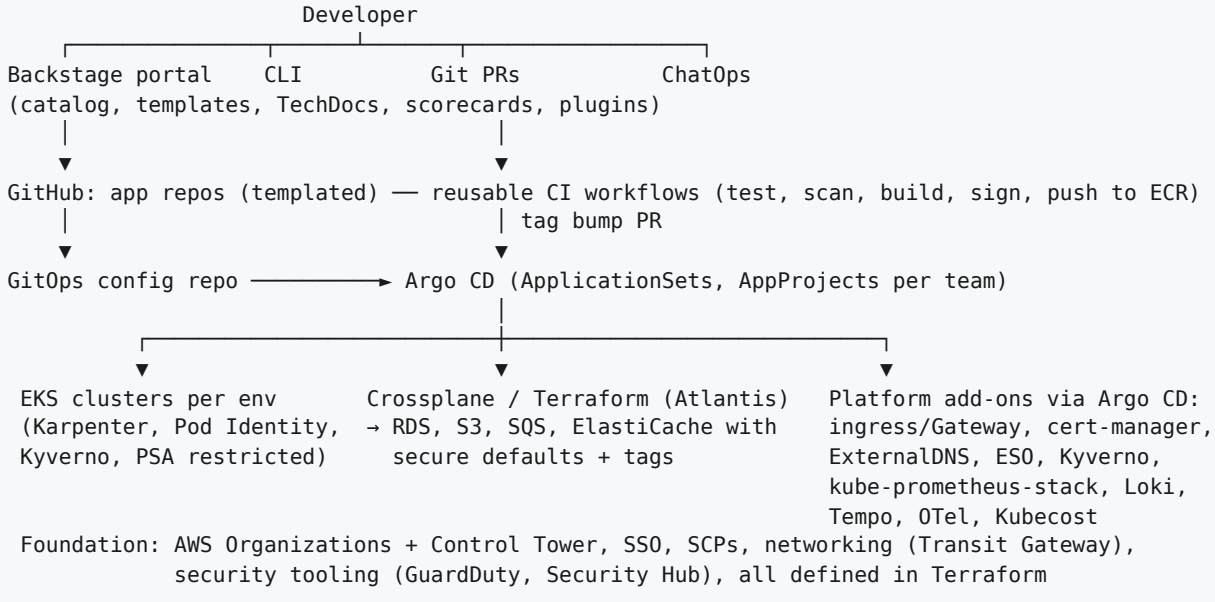
Production readiness scorecard example (shown per service in the portal): owner defined · runbook linked · SLO defined · alerts route to on-call · on supported base image · no critical CVEs older than 7 days · on the latest reusable CI workflow major version · resource requests set · PDB present.

16.9 Measuring the platform

Category	Metrics
Delivery (DORA)	Deployment frequency, lead time for changes, change failure rate, time to restore
Developer experience	Time to first deploy for a new service, time to provision an environment/database, waiting time for tickets, onboarding time for new hires
Adoption	% of services on golden paths / reusable workflows, active portal users, template runs
Satisfaction	Developer surveys (NPS / satisfaction), qualitative interviews (SPACE and DevEx frameworks)
Reliability and efficiency	Platform availability (it's a production system with its own SLOs), cost per service, toil removed

Run the platform like a product: a **roadmap** driven by developer pain points (interviews, surveys, ticket analysis), clear **documentation**, a support channel with SLAs, **release notes** for platform changes, and deprecation policies.

16.10 Reference architecture: an IDP on AWS



16.11 Building a platform: approach and anti-patterns

A pragmatic roadmap

1. **Discover:** interview teams, map the current path to production, measure lead time and pain points (where do they wait? what do they copy-paste?).
2. **Thinnest viable platform:** one golden path for the most common service type: template + reusable pipeline + GitOps deploy + default observability.
3. **Pilot** with one or two friendly teams (collaboration mode); iterate on their feedback.
4. **Productize:** docs, portal/catalog, support model, versioning; measure adoption and time to first deploy.
5. **Expand capabilities** by demand: databases, preview environments, scorecards, cost views.
6. **Mature** (CNCF platform maturity model): from provisional/ad hoc → operational → scalable → optimizing, across investment, adoption, interfaces, operations and measurement.

Anti-patterns

Anti-pattern	Better
Building a portal first ("Backstage project") with nothing behind it	Build the paved road first; add a portal when there's something to expose
Platform as a ticket queue ("ask the platform team for a namespace")	Self-service by default
Mandating the platform without making it better than alternatives	Make the golden path the easiest path; earn adoption
Hiding everything behind a rigid abstraction	Right-level abstractions with escape hatches
Copy-paste templates that drift forever	Centrally versioned reusable workflows, modules and charts
No product management or metrics	Treat as a product: users, roadmap, feedback, adoption metrics
Big-bang migration of all teams	Incremental, starting with new services and willing teams

16.12 Hands-on labs

#	Lab
1	Write a one-page "path to production" map for a sample app: every step, tool and wait time; mark which steps a platform could remove
2	Create a GitHub **template repository** with app skeleton, Dockerfile, tests, CODEOWNERS, Dependabot and `catalog-info.yaml`
3	Turn your CI pipeline into a **reusable workflow** in a separate "platform" repo; call it from two app repos
4	Build a small Helm chart that takes the developer app spec from 16.7 and renders Deployment, Service, HPA, PDB, ServiceMonitor with secure defaults
5	Run Backstage locally (`npx @backstage/create-app@latest`); register your repo via `catalog-info.yaml` ; add TechDocs
6	Add the template from 16.4 to Backstage (with a simple skeleton) and create a new service from it

#	Lab
7	Install Crossplane in kind with a provider (e.g. provider-kubernetes or provider-aws against a lab account with strict cleanup); create an XRD + Composition; claim a resource
8	Add an ApplicationSet pull-request generator to create preview environments per PR in kind
9	Write Kyverno policies enforcing `team` labels and resource requests; show the "cost by team" with OpenCost
10	Define a production readiness scorecard and score three of your lab services manually

16.13 Interview questions

1. What is platform engineering and why has it grown? How does it relate to DevOps and SRE?
2. IDP vs developer portal: what's the difference?
3. What does "platform as a product" mean in daily work?
4. What is a golden path? What makes one successful?
5. Explain Team Topologies and the interaction modes for a platform team.
6. What is cognitive load and how does a platform reduce it?
7. What capabilities would you build first for a 30-team engineering organization, and why?
8. Explain Backstage's catalog, templates and TechDocs. What are the costs of running Backstage?
9. How do developers self-serve infrastructure safely? Terraform modules vs Crossplane vs ACK.
10. How do you choose the right abstraction level? How do you provide escape hatches?
11. How do you roll out updates (e.g. a new security scanner) to 200 services' pipelines?
12. Multi-tenancy options on Kubernetes: namespaces, vcluster, cluster per team: trade-offs.
13. How would you implement ephemeral preview environments per pull request?
14. How do you enforce security, compliance and cost controls without blocking developers?
15. Which metrics prove your platform is working?
16. Describe platform engineering anti-patterns you'd avoid.
17. Design an IDP on AWS/EKS end to end.
18. A team refuses to adopt the platform. What do you do?

16.14 Self-check

- [ ] I can explain IDP, portal, golden path, platform as a product and cognitive load.
- [ ] I can describe Team Topologies and how a platform team should interact with others.
- [ ] I can list the platform capabilities and the interfaces developers use (portal, CLI, API, Git).
- [ ] I can design a golden path for a new service end to end.
- [ ] I can write Backstage catalog entities and a scaffolder template.
- [ ] I can explain self-service infrastructure with Terraform modules and Crossplane.
- [ ] I can compare multi-tenancy models and design preview environments.
- [ ] I can define platform metrics and avoid the common anti-patterns.
- [ ] I can draw a reference IDP architecture on AWS and explain each component.

> **REMEMBER**
>
> - A good platform makes **the right way the easy way**.